



i want to deploy to vercel now via github, how step by step

Perfect! Here's a complete step-by-step guide to deploy your full-stack app. Since you have both frontend (React) and backend (Flask), you'll need to deploy them separately.

▮ Complete Deployment Guide

Part 1: Deploy Frontend (React) to Vercel

Step 1: Prepare Your Project

Create a `.gitignore` file in project root:

```
# Dependencies
node_modules/
.pnp
.pnp.js

# Production
dist/

# Environment variables
.env
.env.local
.env.production.local

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db

# Python
__pycache__/*
*.py[cod]
*.so
.Python
venv/
env/

# Model files
```

```
*.pkl  
models/
```

Step 2: Update .env for Production

Create `.env.example` (to show structure):

```
VITE_RAWG_API_KEY=your_rawg_api_key_here  
VITE_API_URL=https://your-backend-url.com
```

Step 3: Initialize Git Repository

```
git init  
git add .  
git commit -m "Initial commit - Free Game Predictor"
```

Step 4: Push to GitHub

1. Go to github.com and create a new repository
2. Name it: `free-game-predictor` (or your choice)
3. **Don't** initialize with README (you already have files)
4. Copy the commands shown, then run:

```
git remote add origin https://github.com/YOUR_USERNAME/free-game-predictor.git  
git branch -M main  
git push -u origin main
```

Step 5: Deploy to Vercel

1. Go to vercel.com
2. Sign in with GitHub
3. Click **"New Project"**
4. Import your `free-game-predictor` repository
5. Configure:
 - **Framework Preset:** Vite
 - **Root Directory:** `./` (leave default)
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
6. Add **Environment Variables:**
 - `VITE_RAWG_API_KEY`: your RAWG key
 - `VITE_API_URL`: (leave empty for now, we'll add after backend is deployed)

7. Click **"Deploy"**

Wait 2-3 minutes for deployment to complete!

Part 2: Deploy Backend (Flask) to Render

Step 6: Prepare Backend for Deployment

Create `requirements.txt` in `src/` folder:

```
Flask==3.0.0
flask-cors==4.0.0
pandas==2.0.3
numpy==1.24.3
scikit-learn==1.3.0
xgboost==2.0.0
gunicorn==21.2.0
```

Create `src/wsgi.py`:

```
from app import app

if __name__ == "__main__":
    app.run()
```

Step 7: Update `app.py` for Production

Add these lines at the top of `app.py`:

```
import os

# ... existing imports ...

app = Flask(__name__)
CORS(app, origins=["*"]) # Allow all origins for now

# Get port from environment variable for deployment
port = int(os.environ.get('PORT', 5000))

# ... rest of your code ...

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=port)
```

Step 8: Push Backend Changes to GitHub

```
git add .
git commit -m "Prepare backend for deployment"
git push
```

Step 9: Deploy Flask to Render

1. Go to render.com
2. Sign up with GitHub
3. Click **"New +"** → **"Web Service"**
4. Connect your free-game-predictor repository
5. Configure:
 - **Name:** free-game-predictor-api
 - **Root Directory:** src
 - **Environment:** Python 3
 - **Build Command:** pip install -r requirements.txt
 - **Start Command:** gunicorn app:app
6. Click **"Create Web Service"**

Wait 5-10 minutes for deployment!

Step 10: Upload Model Files to Render

Since model files are too large for Git:

1. In Render dashboard, go to your service
2. Click **"Shell"** tab
3. Upload your model files:
 - epic_regression_model.pkl
 - publisher_encoder.pkl
 - publisher_statistics.csv

Or use Render Disk storage for persistent files.

Part 3: Connect Frontend to Backend

Step 11: Update Vercel Environment Variables

1. Go to Vercel dashboard → Your project → **Settings** → **Environment Variables**
2. Add/Update:
 - `VITE_API_URL`: `https://free-game-predictor-api.onrender.com` (your Render URL)
3. Go to **Deployments** → Click "⋮" on latest → **"Redeploy"**

Step 12: Test Your Deployed App! 📄

Visit your Vercel URL (e.g., `free-game-predictor.vercel.app`) and test:

1. Search for a game
2. Click predict
3. Check if prediction works!

📄 Troubleshooting

If prediction doesn't work:

- Check Render logs for Python errors
- Verify model files are uploaded
- Check CORS settings in Flask
- Verify `VITE_API_URL` is correct in Vercel

Frontend build fails:

- Check for any console errors
- Verify all imports are correct
- Make sure `.env` variables are set in Vercel

Your app will be live at:

- **Frontend:** `https://your-project.vercel.app`
- **Backend API:** `https://your-api.onrender.com`

📄 Congratulations on deploying your AI-powered game prediction app!